DEVELOPER GUIDE DG-002

WAFT DESIGN SYSTEM

FOR OFFICIAL USE ONLY

WAFT DESIGN SYSTEM

PRINCIPLES AND PATTERNS

DESIGN PHILOSOPHY

Our design system is built on three core principles:

1. **Clarity Over Cleverness**
2. Code should be obvious, not clever
3. Names should be self-documenting
4. Patterns should be consistent

5. **Simplicity Over Sophistication**

6. Prefer simple solutions
7. Avoid premature optimization
8. Keep it maintainable

9. **Consistency Over Perfection**

10. Consistent patterns are better than perfect patterns
11. Establish conventions and follow them
12. Refactor when patterns emerge

NAMING CONVENTIONS

Functions

- Use verbs: `getUser()`, `calculateTotal()`, `validateInput()`
- Be specific: `getUserById()` not `getUser()`
- Avoid abbreviations: `calculateTotal()` not `calcTot()`

Variables

- Use nouns: `userCount`, `totalPrice`, `isValid`

- Be descriptive: `userCount` not `count`
- Use boolean prefixes: `is`, `has`, `can`, `should`

Classes

- Use nouns: `UserService`, `PaymentProcessor`, `DataValidator`
- Be specific: `UserService` not `Service`
- Use suffixes for types: `UserRepository`, `PaymentController`

CODE ORGANIZATION

File Structure

```
src/
├── models/          # Data models
├── services/       # Business logic
├── controllers/     # Request handling
├── repositories/    # Data access
├── utils/           # Helper functions
└── tests/           # Test files
```

Module Organization

- One class per file (usually)

- Related functions grouped together
- Imports at the top
- Exports clearly defined

ERROR HANDLING

Principles

1. Fail fast
2. Fail clearly
3. Don't swallow errors
4. Log appropriately

Patterns

```
# Good: Explicit error handling
try:
    result = process_data(data)
except ValidationError as e:
    logger.error(f"Validation failed: {e}")
    raise
except ProcessingError as e:
    logger.error(f"Processing failed: {e}")
    return default_value
```

TESTING PATTERNS

Test Structure

- Arrange: Set up test data
- Act: Execute the code
- Assert: Verify the result

Test Naming

- Describe what is being tested
- Include expected outcome
- Example: `test_calculateTotal_returnsSumOfItems()`

DOCUMENTATION STANDARDS

Code Comments

- Explain why, not what
- Keep comments up to date
- Remove dead code and comments

Function Documentation

- Describe purpose
- Document parameters
- Document return value
- Include examples for complex functions

VERSION CONTROL

Commit Messages

- Use present tense: "Add feature" not "Added feature"
- Be specific: "Fix login bug" not "Fix bug"
- Reference issues: "Fix #123: Login bug"

Branch Naming

- Feature: `feature/user-authentication`
- Bug fix: `fix/login-error`
- Hotfix: `hotfix/security-patch`

CONCLUSION

A design system is a living document. It evolves as we learn. The key is consistency and communication.

When in doubt, follow the patterns. When patterns don't exist, create them. When patterns conflict, discuss and decide.